Initiation au Développement : Algorithmique Fondamentale, Variables et Structures de Contrôle

Thème principal : Bases de l'algorithmique, mémoire RAM, variables, affectation, structures conditionnelles et itératives

Synthèse exécutive : Ce cours pose les fondations absolues de la programmation et de la pensée algorithmique en BUT Informatique. Il résout le problème fondamental de la traduction d'un processus logique du monde réel en une suite formelle, séquentielle et déterministe d'instructions exécutables par un ordinateur. La séance traite en détail de la modélisation de la mémoire vive (RAM) via le quadruplet d'une variable (Nom, Type, Valeur, Adresse), de la mécanique de l'affectation destructive, de l'aiguillage du flux d'exécution séquentiel à l'aide de prédicats booléens et de structures conditionnelles (Si / Alors / Sinon), ainsi que de l'automatisation des traitements répétitifs au moyen de structures itératives bornées (Pour) et non bornées (TantQue).

1. Mémorisation et Manipulation des Données : Les Variables et la Mémoire RAM

En informatique, le concept de variable se distingue radicalement de sa définition mathématique statique. Il ne s'agit pas d'une inconnue abstraite dans une équation, mais d'une case mémoire physique concrète située dans la mémoire vive (RAM) de l'ordinateur. Une variable est obligatoirement caractérisée par un quadruplet indissociable : 1. Le Nom (identificateur) : permet au développeur de manipuler la variable dans le code source. 2. Le Type : indique au compilateur le nombre d'octets à réserver et la façon de coder la donnée en binaire. 3. La Valeur : représente le contenu binaire stocké dans la case mémoire à un instant t. 4. L'Adresse mémoire : correspond à la localisation hexadécimale exacte du premier octet de la variable en RAM. Lors de l'exécution d'une instruction d'affectation (par exemple x ← x + 1), le processeur applique un protocole rigoureux en deux étapes chronologiques : - Étape 1 : Il lit en mémoire RAM les valeurs actuelles des variables figurant dans l'expression à droite de l'affectation et évalue le résultat final. - Étape 2 : Il vient inscrire ce résultat binaire dans la case mémoire associée à la variable située à gauche. Ce processus est STRICTEMENT DESTRUCTIF : le nouveau résultat écrase définitivement la valeur qui s'y trouvait auparavant. Un piège classique en examen concerne l'absence d'initialisation : si un algorithme tente de lire une variable n'ayant reçu aucune affectation préalable, le processeur va lire les bits restés en RAM lors des utilisations précédentes par d'autres applications. C'est ce qu'on appelle une 'valeur poubelle', source majeure de dysfonctionnements imprévisibles.

- Variable : Emplacement mémoire étiqueté réservé dans la mémoire vive (RAM) pour stocker une valeur modifiable au cours de l'exécution.
- Quadruplet d'une variable : Ensemble indissociable constitué par son Nom (identificateur), son Type (format et taille en octets), sa Valeur actuelle et son Adresse physique en RAM.
- Typage formel : Spécification explicite du domaine de valeurs (Entier, Réel, Chaîne de caractères, Booléen) déterminant la quantité de mémoire allouée et les opérations autorisées.
- Affectation (← ou =) : Opération fondamentale destructive d'écriture qui évalue l'expression située à droite du symbole et écrase irrémédiablement la valeur stockée à gauche.
- Valeur poubelle : Donnée binaire résiduelle non nettoyée présente en RAM, lue lorsqu'une variable est exploitée sans avoir été préalablement initialisée.

Cas concret enseignant : *La métaphore de l'armoire d'archivage à tiroirs : Imaginez la mémoire RAM comme une grande armoire d'archivage contenant des tiroirs. Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à dimensionner sa profondeur selon son type ('Entier'). L'instruction d'affectation 'solde_compte ← 150' consiste à ouvrir ce tiroir précis, à jeter au recyclage le morceau de papier qui s'y trouvait auparavant (l'ancienne valeur), et à y déposer une nouvelle feuille portant l'inscription 150.*

2. Prise de Décision : Les Structures Conditionnelles (Si / Sinon) et Logique Booléenne

Par défaut, un processeur exécute les instructions d'un programme de façon purement séquentielle, ligne par ligne. Pour introduire du comportement dynamique et adaptatif, on fait appel aux structures conditionnelles. Lors du franchissement d'un bloc 'Si (Condition) Alors [Bloc A] Sinon [Bloc B]', la machine commence par évaluer la condition booléenne : - Si la condition s'évalue à VRAI, le processeur exécute exclusivement les instructions contenues dans le [Bloc A], puis reprend l'exécution après le bloc conditionnel, en ignorant totalement le [Bloc B]. - Si la condition s'évalue à FAUX, le [Bloc A] est sauté et seul le [Bloc B] (s'il est présent) est exécuté. Pour composer des conditions multiples, la maîtrise des tables de vérité est indispensable : - L'opérateur ET (AND) requiert que l'ensemble des sous-conditions soient simultanément vraies pour que la condition globale soit VRAIE. - L'opérateur OU (OR) exige seulement qu'au moins une des sous-conditions soit vraie. - L'opérateur NON (NOT) inverse la valeur de vérité d'une expression.

- • Structure d'Alternative (Si / Alors / Sinon) : Aiguillage logique détournant le flux d'exécution séquentiel selon la valeur de vérité d'une condition.
- • Expression Booléenne (Prédicat) : Proposition logique évaluée par le processeur qui ne peut prendre que deux états strictement binaires : VRAI (True) ou FAUX (False).
- • Opérateurs Logiques (ET, OU, NON) : Connecteurs booléens permettant de composer des conditions complexes régies par leurs tables de vérité respectives.

Cas concret enseignant : *Le cas réel du Distributeur Automatique de Billets (DAB) : Avant d'éjecter l'argent liquide, la machine évalue la condition composée : SI (solde_client >= montant_demandé) ET (billets_disponibles_dans_automate >= montant_demandé). Si cette expression globale s'évalue à VRAI, les billets sont délivrés et le compte est débité ; SINON, l'opération est refusée, un message d'erreur est affiché sur l'écran et la carte bancaire est restituée à l'usager.*

3. Répétition et Automatisation : Les Structures Itératives (Boucles Pour et TantQue)

La puissance de calcul de l'informatique réside dans sa capacité à traiter des opérations répétitives sans défaillance à très haute vitesse. On distingue deux structures itératives fondamentales : 1. La boucle POUR (Structure bornée) : Elle est privilégiée lorsque le nombre d'itérations est connu d'avance. Elle intègre de façon native l'initialisation d'un compteur, le test de limite et l'incrémentation automatique à chaque passage (ex: parcourir les N éléments d'un tableau). 2. La boucle TANT QUE (Structure non bornée) : Elle est employée lorsque le nombre de répétitions dépend d'un événement dynamique (ex: attendre qu'un utilisateur saisisse un mot de passe valide). La condition est évaluée AVANT chaque passage. Si elle est vraie, le corps de la boucle est exécuté ; si elle est fausse dès le départ, le corps de la boucle n'est jamais exécuté. La règle d'or d'une boucle TANT QUE est l'obligation de modifier, à l'intérieur du corps de la boucle, au moins une variable intervenant dans la condition d'arrêt. Si cette variable reste inchangée, la condition demeure éternellement VRAIE, provoquant ce qu'on appelle une 'boucle infinie' qui bloque l'exécution du programme.

- • Boucle Pour (FOR) : Structure itérative bornée et déterministe, utilisée lorsque le nombre précis de répétitions est connu avant le début du traitement.
- • Boucle TantQue (WHILE) : Structure itérative non bornée et dynamique, répétée tant qu'une condition logique demeure VRAIE.
- • Variable de Contrôle & Incrémentation : Modification explicite d'un compteur au sein du corps de la boucle pour garantir le franchissement de la condition d'arrêt.
- • Boucle Infinie : Dysfonctionnement critique lié à l'absence d'actualisation de la condition de sortie dans une boucle TantQue, gelant le programme et monopolisant 100% des ressources CPU.

Cas concret enseignant : *La métaphore des consignes de l'entraîneur d'athlétisme : Si l'entraîneur ordonne au coureur 'Fais 10 tours de piste', il s'agit d'une boucle POUR (le nombre 10 est fixé au départ et la sortie est garantie après 10 itérations). S'il ordonne 'Cours tant qu'il ne pleut pas', il s'agit d'une boucle TANT QUE (la fin de l'exercice dépend d'une condition météo réévaluée à chaque tour).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi l'instruction 'x = x + 1' (ou 'x ← x + 1') est-elle valide en programmation alors qu'elle est mathématiquement impossible ?
Réponse du professeur : En mathématiques, le signe '=' pose une égalité statique (équation sans solution). En informatique, le symbole '=' ou '←' représente un ordre d'affectation dynamique et séquentiel. Le processeur lit d'abord la valeur actuelle de 'x' stockée en RAM à droite (ex: 5), réalise l'addition (5 + 1 = 6), puis vient stocker le résultat 6 dans la case mémoire 'x' située à gauche, ce qui écrase l'ancienne valeur 5.

Question étudiant : Que se passe-t-il concrètement en mémoire vive si j'oublie d'actualiser la variable de contrôle dans une boucle 'TantQue' ?
Réponse du professeur : Si la variable de contrôle n'est pas modifiée dans le corps de la boucle, la condition d'arrêt reste indéfiniment VRAIE. Le programme entre alors dans une 'boucle infinie'. Le processeur réexécute le corps en boucle fermée à 100% de sa puissance, le programme gèle et il devient nécessaire d'interrompre le processus en tuant l'application.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en C++ (respectant strictement les normes de codage de l'IUT d'Anglet) qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme accumulée de tous les entiers compris entre 1 et N inclus (Somme = 1 + 2 + ... + N).

Démarche de résolution :
Étape 1 : En-tête explicite avec cartouche de présentation complet. Étape 2 : Organisation de l'espace de déclaration en respectant la tripartition IUT Anglet (// CONSTANTES, // VARIABLES, // TRAITEMENTS). Étape 3 : Déclaration des variables 'n', 'i' et 'somme' avec le type entier 'int'. Étape 4 : Saisie au clavier du paramètre 'n' annotée par la flèche de flot de données // (clavier) >> saisir >> n. Étape 5 : Initialisation explicite de 'somme = 0' avant d'entrer dans la boucle afin d'écraser toute valeur poubelle en mémoire RAM. Étape 6 : Mise en place d'une boucle 'for' allant de i = 1 jusqu'à n inclus avec un pas de 1 et une indentation de style Allman (accolade ouvrante sur sa propre ligne). Étape 7 : Dans le corps de la boucle, accumulation progressive de 'i' dans 'somme' (somme = somme + i). Étape 8 : Affichage clair du résultat final calculé sur la sortie standard.

```cpp
/*
 * Fichier : somme_entiers.cpp
 * Auteur  : Matéi de Saint Léger
 * Date    : 13/11/2026
 * Cours   : R1.01 - Initiation au Développement
 * Objet   : Calcul de la somme des entiers de 1 à N
 */

#include <iostream>
using namespace std;

int main()
{
    // CONSTANTES

    // VARIABLES
    int n;      // Limite supérieure saisie par l'utilisateur
    int i;      // Compteur d'itérations pour la boucle
    int somme;  // Accumulateur de la somme

    // TRAITEMENTS
    // (clavier) >> saisir >> n
    cout << "Veuillez saisir un nombre entier positif N : ";
    cin >> n;

    // Initialisation stricte pour purger la mémoire RAM de toute valeur poubelle
    somme = 0;

    // Structure itérative bornée
    for (i = 1; i <= n; i++)
    {
        somme = somme + i;
    }

    // afficher >> (écran)
    cout << "La somme des entiers de 1 a " << n << " est egale a : " << somme << endl;

    return 0;
}
```

Résultat : Pour une saisie utilisateur N = 5, les itérations successives sont : i=1 -> somme=1 ; i=2 -> somme=3 ; i=3 -> somme=6 ; i=4 -> somme=10 ; i=5 -> somme=15. Le programme affiche : 'La somme des entiers de 1 a 5 est egale a : 15'. Le résultat est déterministe et conforme aux exigences académiques.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale de critère de choix entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR est choisie lorsque le nombre exact d'itérations est connu à l'avance de manière déterministe (ex: parcourir les N éléments d'un tableau). La boucle TANT QUE est choisie lorsque le nombre de répétitions est indéterminé et dépend de la réalisation dynamique d'une condition (ex: validation d'une saisie utilisateur).

Q : Que valent les variables A et B après l'exécution exacte de la séquence : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?

R : Après A ← 5 et B ← 3, l'instruction A ← A + B attribue à A la valeur 5 + 3 = 8. Puis l'instruction B ← A - B calcule 8 - 3 = 5 et stocke 5 dans B. À la fin de la séquence, A vaut 8 et B vaut 5 (cette séquence réalise l'échange des valeurs de A et B sans variable temporaire).

Q : Pourquoi l'absence d'initialisation d'une variable d'accumulation (ex: somme ← 0) entraîne-t-elle la perte de points en partiel ?

R : Parce qu'une variable non initialisée contient une 'valeur poubelle' (données binaire résiduelles en RAM). Si on effectue 'somme ← somme + i' sans initialiser 'somme' à 0, le premier calcul ajoute 'i' à cette valeur aléatoire, rendant tout le résultat du programme faux.